

Overview:

MyDBReplicableAppGP is a replicated state machine (RSM) built on top of GigaPaxos to provide fault-tolerant database operations. It works with Cassandra to persist data and ensures that all replicas execute operations in a consistent, deterministic order. The main goals of system are to prevent duplicate execution of requests, preserve the correct order of operations, support regular checkpointing, and allow full recovery of state in case of failures.

State Management::

The class uses a ConcurrentHashMap called `executedRequests` to track executed request IDs and prevent duplicate execution. A separate `requestLog` list maintains the order of recent requests up to a defined maximum (`MAX_LOG_SIZE`) for memory management. The system also maintains `latestExecutedSeq` to track the highest sequence number executed. Thread safety is ensured using concurrent data structures and synchronized blocks where necessary, especially during logging and checkpoint operations.

Database Integration:

Cassandra serves as the persistent storage backend for the system. When the application starts, it automatically creates the keyspace if it doesn't already exist and ensures that the grade table is available. The table is designed with a simple schema: an integer primary key (`id`) and a list of integers (`events`). The `executeSQL()` method handles executing queries and includes a built-in recovery mechanism. If a query fails because the table doesn't exist, the system automatically creates the table and retries the query. This design minimizes the need for manual intervention and helps ensure that all state changes are reliably applied.

Request Handling:

Requests arrive as `RequestPacket` instances. SQL statements are extracted either from a JSON "QV" field or directly from the packet value. Before execution, the system checks for duplicates to maintain idempotency. Sequence numbers are parsed from SQL patterns of the form `events+ [<seq>]` to maintain execution order and update `latestExecutedSeq`. Executed requests are logged and old entries are removed when the log exceeds `MAX_LOG_SIZE`.

Checkpointing:

Checkpointing captures the entire system state in a serializable object called `CheckpointState`. This object contains the keyspace name, timestamp, latest executed sequence, executed request IDs, and the complete database state. Checkpoints are serialized to a Base64 string to enable storage or replication. Periodic checkpoints are triggered based on a fixed interval, and on-demand checkpointing is supported as well. The checkpoint mechanism ensures that both in-memory and persistent state can be fully restored after a failure.

Restoration:

The `restore()` method decodes Base64 checkpoint strings and restores all in-memory data structures and Cassandra state. Invalid or empty checkpoints are handled gracefully, starting the system in a fresh state if necessary. This ensures that the system can always recover from crashes or corrupted state, maintaining consistency across replicas.

Testing and Results:

All tests are passed